



**StackMaster Tech**

OBSERVABILITY · SRE · DATA & AI

TRAINING HANDBOOK · DATA ENGINEERING TRACK

# PySpark Fundamentals for Distributed Data Processing

A complete, hands-on handbook — from the Spark engine and architecture to DataFrames, SQL, joins, window functions and real-time streaming with Kafka. Threaded end-to-end through the **SkyOps Analytics** aircraft-telemetry case study.

**9**

Modules

**30+**

Code labs

**9**

Diagrams

**A-Z**

End to end

StackMaster Tech · @StackMasterTech  
Instructor-led & self-paced edition

PySpark Fundamentals  
Participant Handbook

# What you will learn

---

Nine modules building from first principles to a production-style streaming pipeline — each with notes, a diagram, an illustrative UI walkthrough, a real-time SkyOps example and runnable code.

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Apache Spark &amp; PySpark</b>                     | <b>M1</b> |
|          | Why distributed compute exists; what Spark and PySpark are            |           |
| <b>2</b> | <b>PySpark Architecture &amp; Core Components</b>                     | <b>M2</b> |
|          | Driver, executors, cluster manager, DAG and Catalyst                  |           |
| <b>3</b> | <b>Setting Up the PySpark Environment in Skywise</b>                  | <b>M3</b> |
|          | Local, Databricks & Colab setups; the Foundry/Skywise authoring model |           |
| <b>4</b> | <b>PySpark Data Structures: RDDs, DataFrames &amp; Datasets</b>       | <b>M4</b> |
|          | The three abstractions and when to use each                           |           |
| <b>5</b> | <b>PySpark SQL &amp; Structured Data Processing</b>                   | <b>M5</b> |
|          | Temp views, the SQL API, schemas and the Catalyst optimizer           |           |
| <b>6</b> | <b>Data Transformations &amp; Actions in PySpark</b>                  | <b>M6</b> |
|          | Narrow vs wide transformations, lazy evaluation, caching              |           |
| <b>7</b> | <b>Joins, Aggregations &amp; Window Functions</b>                     | <b>M7</b> |
|          | Join strategies, groupBy/agg, and windowed analytics                  |           |
| <b>8</b> | <b>Introduction to Real-Time Data Streaming</b>                       | <b>M8</b> |
|          | Structured Streaming, the unbounded table and micro-batches           |           |
| <b>9</b> | <b>Streaming Architecture &amp; Integration with Kafka</b>            | <b>M9</b> |
|          | End-to-end streaming pipelines, sources, sinks and checkpoints        |           |

# 01

FOUNDATIONS

## Introduction to Apache Spark & PySpark

Why a distributed engine exists, what Spark and PySpark actually are, and where they sit in a modern data platform.

# Introduction to Apache Spark & PySpark

Before writing a single line of PySpark, a learner must understand the problem Spark was built to solve and how Python plugs into a JVM engine. This module sets that mental model.

## Learning objectives

- Explain why single-machine tools break down on big data
- Define Apache Spark and the role of PySpark
- Compare Spark with Hadoop MapReduce
- Place Spark's libraries within the unified engine
- Run a first SparkSession and a tiny DataFrame

## 1.1 The problem: data outgrows one machine

A modern Airbus A320 streams thousands of sensor readings per second. Across a fleet of hundreds of aircraft over months, this becomes terabytes that no single laptop can hold in memory or process in a reasonable time. The core idea of **distributed computing** is to split that data into partitions, spread them across many machines, compute in parallel, and combine the results.

## Pandas vs Spark — when to switch

Pandas is excellent for data that fits in one machine's RAM (roughly up to a few GB). Once data no longer fits, or you need a cluster, Spark's DataFrame API gives you nearly the same ergonomics across many machines. PySpark even ships a `pandas API on Spark` so existing pandas code can scale out.

## 1.2 What is Apache Spark?

Apache Spark is an open-source, **in-memory, distributed compute engine** for large-scale data processing. It keeps intermediate data in memory where possible (rather than writing to disk between every step like classic MapReduce), which makes iterative and interactive workloads dramatically faster. Spark is **unified**: one engine serves batch ETL, SQL analytics, streaming, and machine learning.

## Apache Spark — Unified Analytics Engine

### High-level APIs & Libraries

Spark SQL · Structured Streaming · MLlib · GraphX · pandas API on Spark

### PySpark — Python API

Py4J bridge · DataFrame / RDD / SQL entry points

### Spark Core — RDDs · DAG Scheduler · Catalyst · Tungsten

In-memory distributed compute · fault tolerance · lazy execution

#### Cluster Managers

YARN · Kubernetes · Standalone

#### Storage

HDFS · S3 · Delta · Parquet

#### Languages

Python · Scala · Java · R · SQL

**Figure 1.1.** The Spark stack. Your PySpark code sits on the high-level APIs; everything compiles down to Spark Core running on a cluster manager over distributed storage.

## 1.3 What is PySpark?

**PySpark** is the official Python API for Spark. Spark’s engine runs on the JVM (it is written in Scala). PySpark uses a bridge called **Py4J** so your Python driver code can create and control JVM objects. For DataFrame and SQL operations, your Python instructions are translated into a logical plan that the JVM optimizes and executes — so you get JVM-level performance while writing Python.

### 💡 Why DataFrames are fast in Python

When you use the DataFrame / SQL API, the heavy lifting happens inside the JVM via the Catalyst optimizer — there is almost no per-row Python overhead. Performance only drops when you force data through Python with row-by-row Python UDFs or raw RDD `map()` calls (data must serialize to Python workers and back). Prefer built-in functions.

## 1.4 Spark vs Hadoop MapReduce

| Dimension         | Hadoop MapReduce               | Apache Spark                     |
|-------------------|--------------------------------|----------------------------------|
| Intermediate data | Written to disk between stages | Kept in memory where possible    |
| Speed (iterative) | Slow (disk I/O each step)      | 10–100× faster on many workloads |
| Programming model | Verbose map/reduce             | High-level DataFrame, SQL, RDD   |
| Workloads         | Batch only                     | Batch, SQL, streaming, ML, graph |
| Languages         | Mostly Java                    | Python, Scala, Java, R, SQL      |

### ✈️ Real-time context — Airbus Skywise

Skywise is Airbus’s aviation data platform, built on Palantir Foundry, that ingests operational and sensor data from thousands of aircraft across many airlines. At that scale, distributed engines like Spark are what make fleet-wide analytics, predictive maintenance, and fuel-efficiency studies feasible. Throughout this handbook we use a running **SkyOps Analytics** example — aircraft telemetry — so every concept maps to something concrete.

## 1.5 Your first PySpark program

Every PySpark application begins by creating a `SparkSession` — the single entry point to all functionality.

PYSPARK / PYTHON

```
from pyspark.sql import SparkSession

# The SparkSession is the entry point to Spark functionality
spark = (SparkSession.builder
         .appName("SkyOpsAnalytics")
         .getOrCreate())

# Create a tiny DataFrame from Python data
data = [("A320-001", 38000, 451),
        ("A320-002", 41000, 462),
        ("A350-010", 39000, 488)]
columns = ["aircraft_id", "altitude_ft", "airspeed_kts"]

df = spark.createDataFrame(data, columns)
df.show()
print("Rows:", df.count())
```

### CONSOLE OUTPUT

```
+-----+-----+-----+
|aircraft_id|altitude_ft|airspeed_kts|
+-----+-----+-----+
|  A320-001|      38000|         451|
|  A320-002|      41000|         462|
|  A350-010|      39000|         488|
+-----+-----+-----+
Rows: 3
```



### Lab 1.1 — Hello, Spark

Install PySpark locally (Module 3), open a Python shell, and reproduce the program above. Change the data, add a column `fuel_flow_kgph`, and re-run `df.show()`. Confirm `spark.version` prints your Spark version.

## 1.6 Where Spark is used

### ETL / data engineering

Clean and join terabytes into curated tables.

### Analytics & BI

SQL over huge datasets feeding dashboards.

### Streaming

Near real-time pipelines from Kafka and sensors.

### Machine learning

MLlib for feature engineering and models at scale.

### Predictive maintenance

Fleet telemetry → failure prediction.

### Lakehouse

Delta Lake tables with ACID guarantees.

### 🔑 Key takeaways

- Spark is an in-memory, distributed, **unified** compute engine.
- PySpark is the Python API; it bridges to the JVM engine via Py4J.
- Prefer DataFrame/SQL APIs — they run in the JVM and avoid Python overhead.
- Spark beats MapReduce mainly by avoiding disk I/O between stages.
- Our running example is aircraft telemetry (SkyOps Analytics).

# 02

INTERNALS

## PySpark Architecture & Core Components

Driver, executors, cluster manager, the DAG, Catalyst and Tungsten — how a line of PySpark becomes parallel work.





# PySpark Architecture & Core Components

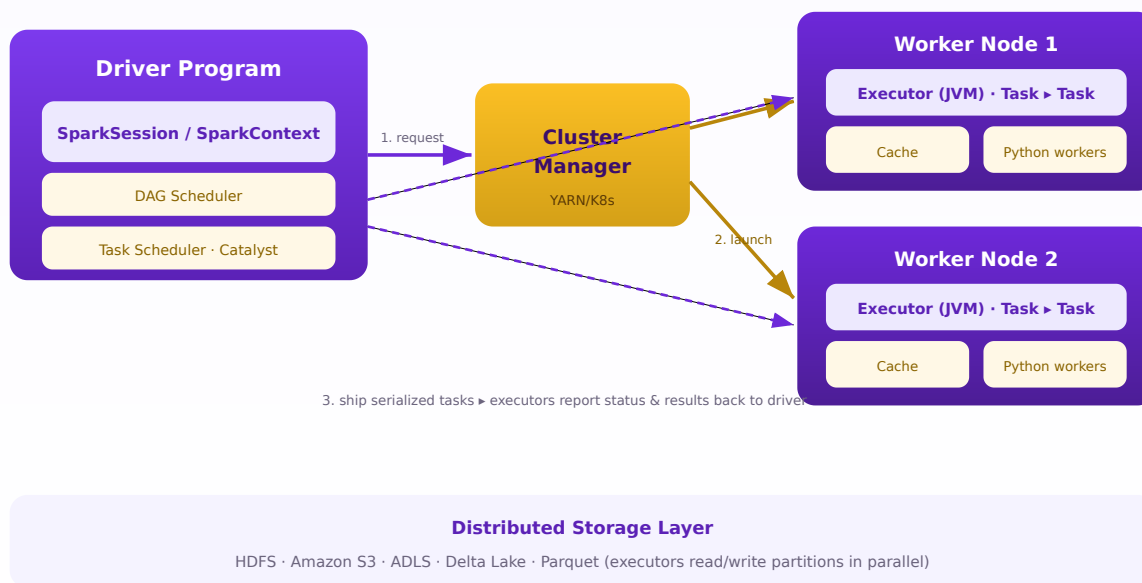
Understanding the runtime is what separates someone who writes PySpark from someone who can debug and tune it. This module maps the moving parts.

## 🎯 Learning objectives

- Identify the driver, executors and cluster manager
- Describe partitions and parallelism
- Explain lazy evaluation and the DAG
- Trace how a job is split into stages and tasks
- Read the Spark Web UI to inspect a run

## 2.1 The cluster at a glance

A Spark application has one **Driver** (your program, holding the `SparkSession`) and many **Executors** (JVM processes on worker nodes that run tasks and cache data). A **Cluster Manager** (YARN, Kubernetes, or Spark Standalone) allocates the executors. The driver plans the work; executors do the work.



**Figure 2.1.** Spark runtime architecture. The driver requests resources from the cluster manager, which launches executors; the driver then ships serialized tasks to executors that read/write storage in parallel.

| Component       | Runs where              | Responsibility                                       |
|-----------------|-------------------------|------------------------------------------------------|
| Driver          | One process (your app)  | Builds DAG, schedules stages/tasks, collects results |
| SparkSession    | Inside driver           | Single entry point; wraps SparkContext               |
| Cluster Manager | Cluster control plane   | Allocates executors (YARN/K8s/Standalone)            |
| Executor        | Worker nodes            | Runs tasks, holds cached partitions in memory        |
| Task            | One core on an executor | Processes one partition of data                      |

## 2.2 Job → Stage → Task

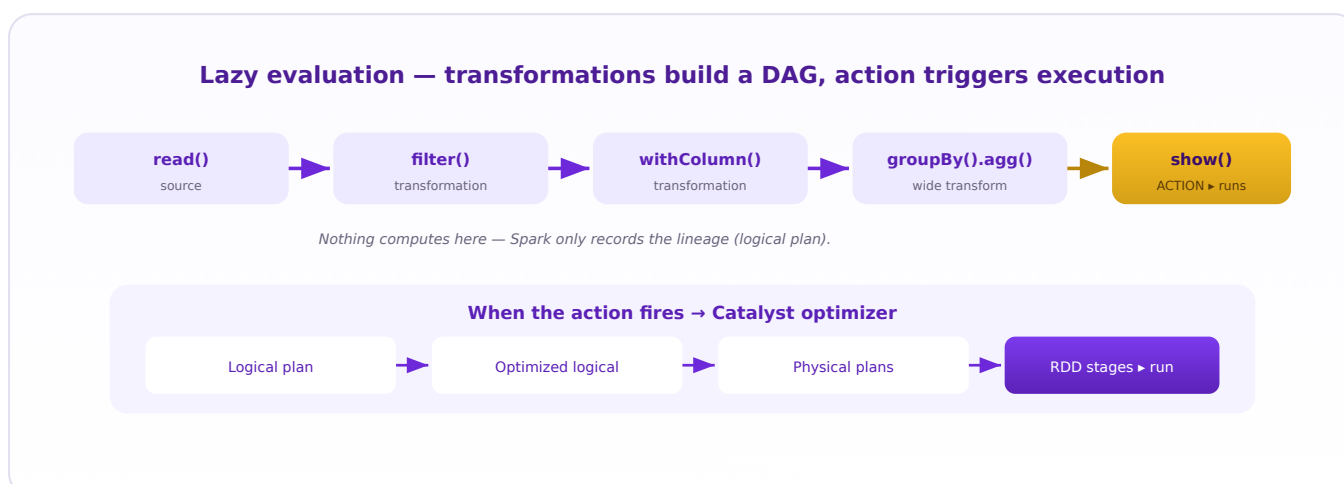
When an **action** runs, Spark creates a **Job**. The job is split into **Stages** at shuffle boundaries (wide transformations). Each stage runs as a set of **Tasks** — one task per data **partition**. If you have 200 partitions and 8 cores, Spark runs 8 tasks at a time until all 200 finish.

### Partitions = the unit of parallelism

More partitions → more parallelism but more scheduling overhead; too few → idle cores and skew. A common default for shuffles is 200 ( `spark.sql.shuffle.partitions` ). Tune it to your data size and cluster cores.

## 2.3 Lazy evaluation and the DAG

PySpark **transformations** (e.g. `filter`, `select`, `groupBy`) are **lazy**: they only record what you want, building a Directed Acyclic Graph (DAG) of operations. Nothing executes until an **action** (e.g. `show`, `count`, `write`) is called. This lets the **Catalyst** optimizer rearrange and prune the plan before any data moves.



**Figure 2.2.** Lazy evaluation. Transformations only build a logical plan; the action triggers Catalyst optimization and physical execution.

## Catalyst & Tungsten

**Catalyst** is Spark's query optimizer: it turns your DataFrame operations into an optimized physical plan (predicate pushdown, column pruning, join reordering). **Tungsten** is the execution engine that uses off-heap memory and code generation for CPU-efficient processing. Together they are why DataFrames outperform hand-written RDD code.

## 2.4 Inspecting the plan

Use `explain()` to see how Spark will execute a query, and partition methods to see parallelism.

```
df = spark.read.parquet("/raw/aircraft_telemetry")

# Number of partitions backing this DataFrame
print("Partitions:", df.rdd.getNumPartitions())

# See the optimized physical plan (no data is moved yet – still lazy)
busy = df.filter(df.altitude_ft > 35000).select("aircraft_id", "altitude_ft")
busy.explain(mode="formatted")
```

#### CONSOLE OUTPUT

```
Partitions: 8
== Physical Plan ==
* Project (3)
+- * Filter (2)
   +- * ColumnarToRow (1)
      +- Scan parquet /raw/aircraft_telemetry
         PushedFilters: [IsNotNull(altitude_ft), GreaterThan(altitude_ft,35000)]
```

#### 💡 Read the plan bottom-up

Notice **PushedFilters**: Catalyst pushed the altitude filter down into the Parquet scan, so Spark skips reading row-groups that can't match. Reading `explain()` is the single most useful tuning skill.

## 2.5 The Spark Web UI

Every running application exposes a Web UI (default port `4040`) showing jobs, stages, tasks, storage and executors. It is your primary debugging tool.

The screenshot shows the Spark Web UI interface. The address bar displays `http://<driver-host>:4040/jobs/`. The top navigation bar includes links for Spark 3.5, Jobs, Stages, Storage, Environment, Executors, and SQL. The current application is identified as 'SkyOpsAnalytics · application\_1732\_0007'. The main section is titled 'Completed Jobs (3)' and contains a table with the following data:

| Job Id | Description                      | Stages | Tasks   | Duration |
|--------|----------------------------------|--------|---------|----------|
| 2      | show at fleet_kpis.py:48         | 2/2    | 16/16   | 0.9 s    |
| 1      | groupBy/agg at fleet_kpis.py:40  | 2/2    | 208/208 | 3.4 s    |
| 0      | parquet read at fleet_kpis.py:22 | 1/1    | 8/8     | 1.1 s    |

Below the table is an event timeline bar with a legend: 'Event timeline — green = compute, gold = shuffle write'.

**Screenshot 2.1.** Spark Web UI — Jobs tab. Each action becomes a job; the groupBy job shows 208 tasks (200 shuffle partitions + reads).

http://<driver-host>:4040/executors/

**Spark 3.5** Executors

| Executor ID   | Cores | Active Tasks | Storage Memory  | Input  | Shuffle R/W     |
|---------------|-------|--------------|-----------------|--------|-----------------|
| <b>driver</b> | —     | 0            | 0 / 2.1 GB      | 0 B    | 0 / 0           |
| 1             | 4     | 4            | 512 MB / 4.6 GB | 1.2 GB | 320 MB / 410 MB |
| 2             | 4     | 4            | 498 MB / 4.6 GB | 1.1 GB | 300 MB / 395 MB |

2 executors × 4 cores = 8 parallel tasks. Watch **Shuffle** columns to spot wide-transformation cost.

**Screenshot 2.2.** Executors tab. Two executors × 4 cores give 8 parallel tasks; the Shuffle Read/Write columns reveal wide-transformation cost.

### 🔑 Key takeaways

- Driver plans; executors execute; cluster manager allocates.
- A job splits into stages at shuffle boundaries; each stage = one task per partition.
- Transformations are lazy; actions trigger execution.
- Catalyst optimizes the plan; Tungsten executes it efficiently.
- `explain()` and the Web UI (:4040) are your core debugging tools.

# 03

## ENVIRONMENT

# Setting Up the PySpark Environment in Skywise

A portable local setup, a free cloud notebook, and the Foundry-based Skywise path — plus how to verify each.



# Setting Up the PySpark Environment in Skywise

Learners need a working environment first. Because Skywise is a client-internal platform, we cover a portable setup learners can keep, plus exactly how PySpark is authored inside Skywise/Foundry.

## Learning objectives

- Install and verify PySpark locally
- Use a free cloud notebook for labs
- Understand the Skywise / Foundry authoring model
- Configure a SparkSession with sensible defaults
- Validate the environment end to end

## About “Skywise” in this module

Skywise is Airbus’s aviation data platform, built on **Palantir Foundry**. Its specific workspaces, dataset paths and access are **client-internal** and provisioned by the client — they cannot be installed publicly. So this module teaches (a) a portable local/cloud environment every learner can keep, and (b) the **authoring model** used inside Foundry/Skywise so the transition is seamless. Confirm the exact Skywise project paths and access with the client before the session.

## 3.1 Option A — Local install (recommended for learning)

The fastest sandbox is a local pip install. It needs Java 8/11/17 and Python 3.8+.

SHELL

```
# 1) Check Java is present (Spark runs on the JVM)
java -version

# 2) Create an isolated environment and install PySpark
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install pyspark==3.5.1 jupyter

# 3) Launch a notebook or the interactive shell
pyspark                          # opens the PySpark REPL
# or
jupyter notebook
```

## Windows winutils note

On Windows, set `HADOOP_HOME` and place a matching `winutils.exe` on the path to avoid Hadoop IO warnings. On macOS/Linux this is not needed.

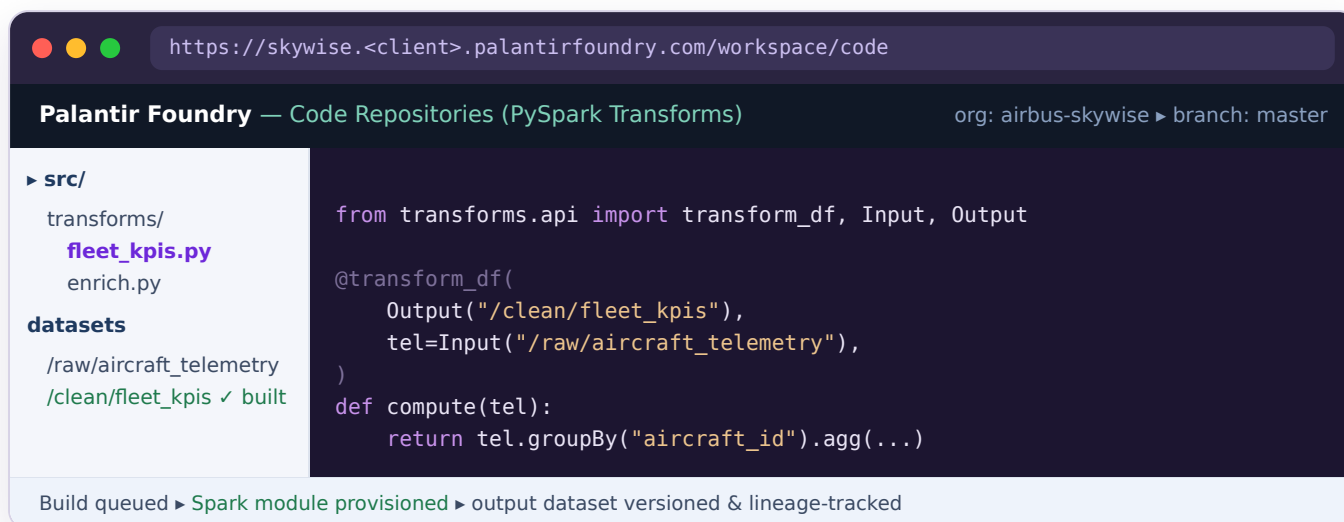
## 3.2 Option B — Free cloud notebook

For zero local setup, use a managed notebook. **Databricks Free Edition** is the current option for self-learning (the older Databricks *Community Edition* was retired in January 2026). Google Colab also works with a quick `pip install pyspark`.

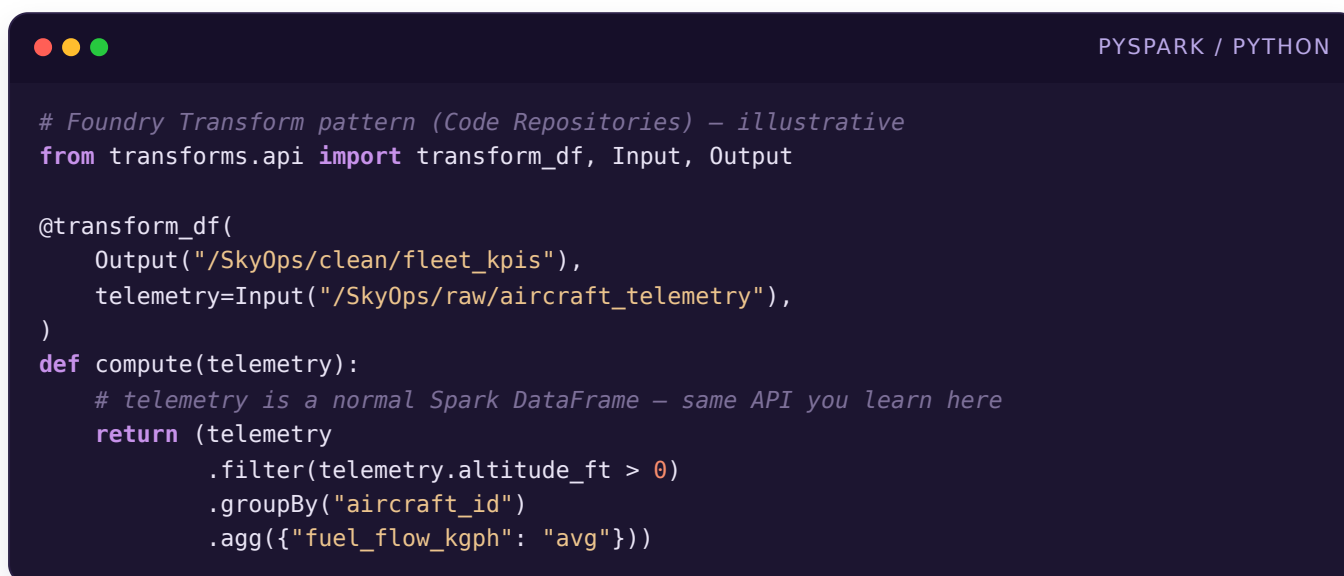
| Environment             | Setup effort       | Best for                                 |
|-------------------------|--------------------|------------------------------------------|
| Local pip install       | Low                | Offline labs, full control               |
| Databricks Free Edition | None (browser)     | Realistic notebook + Spark UI            |
| Google Colab + pyspark  | One cell           | Quick demos, sharing                     |
| Skywise / Foundry       | Client-provisioned | Production transforms on real fleet data |

### 3.3 Option C — Authoring PySpark in Skywise / Foundry

Inside Skywise (Foundry), you do not manage clusters yourself. You write PySpark in two places: **Code Workbook** (interactive, notebook-like, great for exploration) and **Code Repositories** (git-backed production **Transforms**). A Transform declares typed **Inputs** and **Outputs** (datasets), and Foundry provisions a Spark module to run it, versioning the output and tracking lineage automatically.



**Screenshot 3.1.** Illustrative Foundry Code Repositories view. A PySpark Transform reads a raw dataset and writes a clean, lineage-tracked output. Exact paths/UI vary by client.



## The skill transfers 1:1

Whether you run locally, in Databricks, or in a Foundry Transform, the DataFrame API is identical. Everything in Modules 4–9 works the same way; only the entry point (a local `SparkSession` vs Foundry-managed inputs) differs.

## 3.4 Configuring the SparkSession

For local/cloud setups, build the session with a few sensible defaults.

```
PYSPARK / PYTHON

from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("SkyOpsAnalytics")
        .master("local[*]")                # use all local cores
        .config("spark.sql.shuffle.partitions", "8") # small for laptops
        .config("spark.sql.adaptive.enabled", "true") # AQE: auto-tune shuffles
        .getOrCreate())

spark.sparkContext.setLogLevel("WARN") # quieter logs
print("Spark:", spark.version)
print("Cores:", spark.sparkContext.defaultParallelism)
```

### CONSOLE OUTPUT

```
Spark: 3.5.1
Cores: 8
```

## 3.5 Verify end to end

```
PYSPARK / PYTHON

# A 5-line smoke test that exercises read, transform and action
df = spark.range(0, 1_000_000)
result = df.filter(df.id % 7 == 0).count()
print("Multiples of 7:", result) # 142858
print("Web UI:", spark.sparkContext.uiWebUrl) # open this in a browser
```

### CONSOLE OUTPUT

```
Multiples of 7: 142858
Web UI: http://localhost:4040
```

## Lab 3.1 — Stand up your environment

Pick Option A or B, create the `SparkSession` above, run the smoke test, then open the Web UI at :4040 and find the count job under the Jobs tab. You now have everything needed for the rest of the handbook.



### 🔑 Key takeaways

- Local pip install is the simplest portable sandbox; cloud notebooks need zero setup.
- Databricks **Free Edition** replaced the retired Community Edition (Jan 2026).
- Skywise = Airbus on Foundry; author PySpark via Code Workbook & Code Repositories (Transforms).
- The DataFrame API is identical across local, cloud and Foundry.
- Always verify with a smoke test and the Web UI before teaching.

# 04

DATA STRUCTURES

## RDDs, DataFrames & Datasets

The three Spark abstractions, when to use each, schemas, and why DataFrames are the PySpark default.



# Understanding PySpark Data Structures: RDDs, DataFrames & Datasets

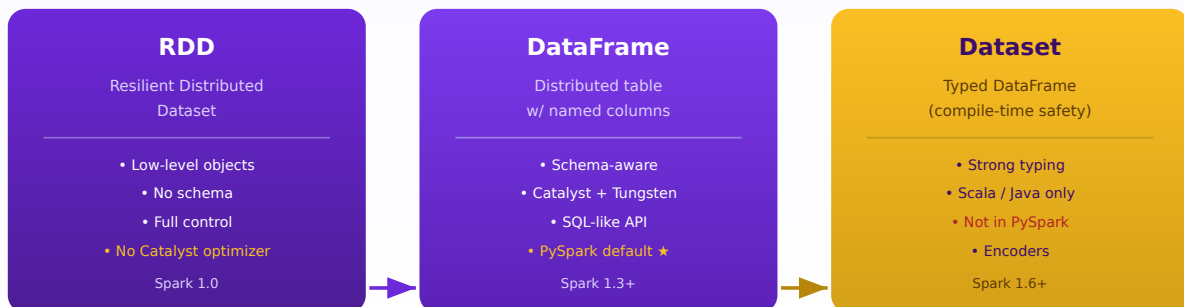
Spark offers three abstractions. Knowing their trade-offs prevents the most common PySpark performance mistakes.

## 🎯 Learning objectives

- Distinguish RDDs, DataFrames and Datasets
- Explain why DataFrames are the PySpark default
- Define and apply an explicit schema
- Convert between RDDs and DataFrames
- Decide when a low-level RDD is justified

## 4.1 Three abstractions, increasing structure

### Three Spark abstractions — increasing optimization & structure



**Figure 4.1.** RDD → DataFrame → Dataset. Structure and optimization increase left to right. Datasets are typed and Scala/Java-only; in PySpark you work with DataFrames.

|                      | RDD                         | DataFrame           | Dataset                  |
|----------------------|-----------------------------|---------------------|--------------------------|
| Schema               | None                        | Yes (named columns) | Yes + compile-time types |
| Optimizer            | No Catalyst                 | Catalyst + Tungsten | Catalyst + Tungsten      |
| Languages            | All                         | All                 | Scala / Java only        |
| Available in PySpark | Yes                         | Yes (default)       | <b>No</b>                |
| Best for             | Unstructured / fine control | Most work           | Typed JVM apps           |

## 🔗 In PySpark, “Dataset = DataFrame”

Because Python is dynamically typed, the typed Dataset API exists only in Scala/Java. In PySpark a DataFrame *is* the untyped Dataset ( `Dataset[Row]` ). So in practice your decision is just **RDD vs DataFrame** — and the answer is almost always DataFrame.

## 4.2 RDD — the low-level foundation

An **RDD** (Resilient Distributed Dataset) is an immutable, partitioned collection of objects with no schema. It is the original Spark abstraction. You rarely need it directly today, but it underpins everything.

PYSPARK / PYTHON

```
sc = spark.sparkContext
rdd = sc.parallelize(["A320-001,38000", "A320-002,41000", "A350-010,39000"])

# Parse each line and keep flights above 40,000 ft
high = (rdd
    .map(lambda line: line.split(","))
    .map(lambda parts: (parts[0], int(parts[1])))
    .filter(lambda t: t[1] > 40000))

print(high.collect())
```

#### CONSOLE OUTPUT

```
[('A320-002', 41000)]
```

#### ⚠ Why not RDDs by default

RDD code is opaque to Catalyst — Spark can't optimize your lambdas, and in PySpark every element crosses the Python↔JVM boundary. Reserve RDDs for genuinely unstructured data or custom partitioning. For everything else, DataFrames are faster and clearer.

## 4.3 DataFrame — the workhorse

A **DataFrame** is a distributed table with named, typed columns. It exposes a rich API and gets the full Catalyst/Tungsten optimization. This is what you will use 95% of the time.

PYSPARK / PYTHON

```
from pyspark.sql import Row

rows = [Row(aircraft_id="A320-001", altitude_ft=38000, status="cruise"),
        Row(aircraft_id="A320-002", altitude_ft=41000, status="cruise"),
        Row(aircraft_id="A350-010", altitude_ft=2200, status="approach")]

df = spark.createDataFrame(rows)
df.printSchema()
df.show()
```

#### CONSOLE OUTPUT

```
root
|-- aircraft_id: string (nullable = true)
|-- altitude_ft: long (nullable = true)
|-- status: string (nullable = true)

+-----+-----+-----+
|aircraft_id|altitude_ft|  status|
+-----+-----+-----+
|   A320-001|      38000|  cruise|
|   A320-002|      41000|  cruise|
|   A350-010|       2200|approach|
+-----+-----+-----+
```

## 4.4 Schemas — let Spark know the shape

Spark can infer a schema, but for reliability and speed (especially with CSV/JSON) define it explicitly with `StructType`.

PYSPARK / PYTHON

```
from pyspark.sql.types import (StructType, StructField,
                                StringType, IntegerType, DoubleType, TimestampType)

schema = StructType([
    StructField("aircraft_id", StringType(), False),
    StructField("ts", TimestampType(), True),
    StructField("altitude_ft", IntegerType(), True),
    StructField("fuel_flow_kgph", DoubleType(), True),
])

telemetry = (spark.read
             .schema(schema)           # explicit schema = no inference scan
             .option("header", True)
             .csv("/raw/aircraft_telemetry.csv"))
telemetry.printSchema()
```

### 💡 Explicit schema = faster + safer

Inferring a schema makes Spark read part of the file twice and can guess wrong types. An explicit `StructType` avoids that scan and guarantees the contract your downstream code expects.

## 4.5 Converting between RDD and DataFrame

PYSPARK / PYTHON

```
# DataFrame -> RDD (drop to row objects)
rdd_again = telemetry.rdd
print(rdd_again.first())

# RDD -> DataFrame (attach a schema)
back_to_df = rdd_again.toDF()
```

## SkyOps — choosing the abstraction

Raw fleet sensor logs arrive as semi-structured text from many aircraft. We do a thin RDD pass only if we must repair malformed lines; otherwise we read straight into a typed DataFrame and stay there for every downstream KPI. That single decision is the difference between a job that finishes in minutes and one that crawls.

### Key takeaways

- Three abstractions: RDD (low-level), DataFrame (default), Dataset (Scala/Java only).
- In PySpark, DataFrame = untyped Dataset[Row]; the real choice is RDD vs DataFrame.
- DataFrames get Catalyst/Tungsten optimization; RDDs do not.
- Define explicit schemas with `StructType` for speed and safety.
- Drop to RDD only for unstructured data or custom partitioning.

# 05

SQL

## PySpark SQL & Structured Data Processing

Read structured data, register views, run SQL and DataFrame queries interchangeably, and master the column API.



# Introduction to PySpark SQL & Structured Data Processing

Spark SQL lets analysts and engineers meet in the middle: the same engine answers a SQL string or a chained DataFrame expression with identical performance.

## Learning objectives

- Read Parquet, CSV and JSON into DataFrames
- Register a DataFrame as a temp view and query with SQL
- Use the column expression API (select, filter, withColumn)
- Apply built-in functions instead of UDFs
- Write results back to storage

## 5.1 Reading structured data

PYSPARK / PYTHON

```
# Parquet is columnar and self-describing – the preferred format
df = spark.read.parquet("/raw/aircraft_telemetry")

# CSV with header + schema
csv_df = (spark.read.option("header", True)
          .schema(schema)
          .csv("/raw/telemetry.csv"))

# JSON (one object per line)
json_df = spark.read.json("/raw/events.json")
```

### Prefer Parquet/Delta over CSV

Columnar formats let Catalyst prune columns and push filters, reading only what a query needs. CSV forces full scans. For anything beyond a small import, store curated data as Parquet or Delta.

## 5.2 SQL and DataFrames are interchangeable

Register any DataFrame as a temporary view, then query it with standard SQL. Both paths compile to the same optimized plan.



```

df.createOrReplaceTempView("telemetry")

# (a) Pure SQL
sql_result = spark.sql("""
    SELECT aircraft_id,
           ROUND(AVG(fuel_flow_kgph), 1) AS avg_fuel
    FROM telemetry
    WHERE altitude_ft > 30000
    GROUP BY aircraft_id
    ORDER BY avg_fuel DESC
""")

# (b) Equivalent DataFrame API
from pyspark.sql import functions as F
df_result = (df.filter(F.col("altitude_ft") > 30000)
             .groupBy("aircraft_id")
             .agg(F.round(F.avg("fuel_flow_kgph"), 1).alias("avg_fuel"))
             .orderBy(F.desc("avg_fuel")))

sql_result.show()

```

#### CONSOLE OUTPUT

```

+-----+-----+
|aircraft_id|avg_fuel|
+-----+-----+
|  A350-010|  2410.6|
|  A320-002|  1980.3|
|  A320-001|  1875.9|
+-----+-----+

```

## 5.3 The column expression API

Most DataFrame work is composing column expressions with `F.col()` and the functions module. Learn these four verbs and you can express almost anything.

| Verb                                     | Purpose                  | Example                                                          |
|------------------------------------------|--------------------------|------------------------------------------------------------------|
| <code>select</code>                      | Choose / compute columns | <code>df.select("aircraft_id", F.col("altitude_ft")/3.28)</code> |
| <code>filter</code> / <code>where</code> | Keep matching rows       | <code>df.filter(F.col("status")==="cruise")</code>               |
| <code>withColumn</code>                  | Add / replace a column   | <code>df.withColumn("alt_m", F.col("altitude_ft")*0.3048)</code> |
| <code>withColumnRenamed</code>           | Rename                   | <code>df.withColumnRenamed("ts", "event_time")</code>            |

```

from pyspark.sql import functions as F

enriched = (df
    .withColumn("altitude_m", F.round(F.col("altitude_ft") * 0.3048, 0))
    .withColumn("phase", F.when(F.col("altitude_ft") > 30000, "cruise")
                          .when(F.col("altitude_ft") > 10000, "climb_descent")
                          .otherwise("ground_or_approach"))
    .select("aircraft_id", "ts", "altitude_m", "phase"))
enriched.show(3)

```

#### CONSOLE OUTPUT

```

+-----+-----+-----+-----+
|aircraft_id|          ts|altitude_m| phase|
+-----+-----+-----+-----+
|  A320-001|2026-06-01 08:00:00|   11582.0|cruise|
|  A320-001|2026-06-01 08:05:00|   12040.0|cruise|
|  A350-010|2026-06-01 08:00:00|    670.0|ground_or_approach|
+-----+-----+-----+-----+

```

## 5.4 Built-in functions beat UDFs

The `pyspark.sql.functions` module has hundreds of optimized functions (string, date, math, conditional, array). Use them before writing a Python UDF, which is slower because rows cross into Python.

### ⚠ UDFs are a last resort

A Python UDF serializes every row to a Python worker, runs your function, and serializes back — bypassing Tungsten. If you must use one, prefer a **pandas UDF** (vectorized) over a plain UDF. Always check for a built-in first.

## 5.5 Writing results

```

(df_result
    .write
    .mode("overwrite")          # overwrite | append | ignore | error
    .partitionBy("aircraft_id") # physical layout for fast filtered reads
    .parquet("/clean/fuel_by_aircraft"))

```

### 🔧 Lab 5.1 — SQL vs DataFrame parity

Load the telemetry Parquet, then compute average airspeed per aircraft two ways — once with `spark.sql()` and once with the DataFrame API — and confirm the outputs match. Run `.explain()` on both and observe the plans are essentially identical.

### 🔑 Key takeaways

- Spark SQL and the DataFrame API compile to the same optimized plan.
- Register temp views to mix SQL and DataFrame freely.
- Master `select / filter / withColumn` + the `functions` module.
- Prefer built-ins over UDFs; if needed, use vectorized pandas UDFs.
- Store curated output as partitioned Parquet/Delta.

# 06

COMPUTE

## Transformations & Actions in PySpark

Narrow vs wide transformations, lazy chains, the shuffle, caching, and the actions that trigger real work.



# Data Transformations & Actions in PySpark

This is the mental model that determines performance: what is lazy, what triggers execution, and what forces an expensive shuffle.

## 🎯 Learning objectives

- Separate transformations from actions
- Distinguish narrow from wide transformations
- Explain the shuffle and its cost
- Use cache/persist correctly
- Avoid the common 'action in a loop' trap

## 6.1 Transformations vs actions

**Transformations** return a new DataFrame and are lazy. **Actions** return a value to the driver (or write data) and trigger execution of the whole lineage.

### Transformations (lazy)

| Function                                         | Type   |
|--------------------------------------------------|--------|
| <code>select</code> , <code>filter</code>        | narrow |
| <code>withColumn</code> , <code>map</code>       | narrow |
| <code>groupBy</code> , <code>join</code>         | wide   |
| <code>distinct</code> , <code>repartition</code> | wide   |
| <code>orderBy</code>                             | wide   |

### Actions (eager)

| Function                                    | Returns            |
|---------------------------------------------|--------------------|
| <code>show()</code>                         | prints rows        |
| <code>count()</code>                        | row count          |
| <code>collect()</code>                      | all rows to driver |
| <code>take(n)</code> / <code>first()</code> | n rows             |
| <code>write...()</code>                     | persists data      |

### ⚠️ `collect()` can crash your driver

`collect()` pulls *all* rows into the driver's memory. On big data this triggers an out-of-memory crash. Use `show()`, `take(n)`, or write to storage instead.

## 6.2 Narrow vs wide — and the shuffle

A **narrow** transformation maps each input partition to one output partition with no data movement ( `filter`, `select` ). A **wide** transformation must redistribute data across the network by key — the **shuffle** — which writes to disk and crosses machines ( `groupBy`, `join`, `distinct` ).



**Figure 6.1.** Narrow transformations stay within a partition; wide transformations shuffle data across the network, creating a new stage boundary.

### 💡 Minimize and delay shuffles

Filter and select *early* to shrink data before any shuffle. Each wide transformation starts a new stage; fewer shuffles means fewer stages and faster jobs. Spark's Adaptive Query Execution (AQE) can also coalesce shuffle partitions automatically — keep

```
spark.sql.adaptive.enabled=true
```

## 6.3 A lazy chain, then one action

```

PYSPARK / PYTHON

from pyspark.sql import functions as F

# All transformations below are lazy – they build a plan, nothing runs yet
pipeline = (spark.read.parquet("/raw/aircraft_telemetry")
            .filter(F.col("altitude_ft") > 30000)           # narrow
            .withColumn("hour", F.hour("ts"))             # narrow
            .groupBy("aircraft_id", "hour")                # wide (shuffle)
            .agg(F.avg("fuel_flow_kgph").alias("avg_fuel"),
                 F.max("engine_temp_c").alias("max_temp")))

# The action below triggers the whole pipeline to execute
pipeline.show(5)    # ACTION

```

### CONSOLE OUTPUT

```

+-----+-----+-----+-----+
|aircraft_id|hour|avg_fuel|max_temp|
+-----+-----+-----+-----+
|  A320-001|   8|  1875.9|   612|
|  A320-002|   8|  1980.3|   628|
|  A350-010|   9|  2410.6|   655|
+-----+-----+-----+-----+

```

## 6.4 Caching — reuse without recompute

Because Spark is lazy, calling two actions on the same chain recomputes it twice. If you reuse a DataFrame, `cache()` (or `persist()`) keeps it in memory after the first action.

```

busy = (spark.read.parquet("/raw/aircraft_telemetry")
        .filter(F.col("altitude_ft") > 30000))

busy.cache()           # mark for caching (lazy until first action)
print(busy.count())     # action 1 -> computes AND caches
print(busy.agg(F.avg("airspeed_kts")).first()) # action 2 -> reuses cache

busy.unpersist()       # free memory when done

```

### ✂ When caching helps (and hurts)

Cache only DataFrames you reuse **multiple times** and that fit in memory. Caching a once-used DataFrame just wastes memory. Check the Storage tab in the Web UI to confirm what is cached.

### ✈ SkyOps — one read, many KPIs

Our fleet job filters telemetry to cruise altitude once, caches it, then derives several KPIs (avg fuel, max engine temperature, anomaly counts) from that cached frame. Without the cache, each KPI would re-read and re-filter terabytes; with it, the expensive scan happens exactly once.

## 6.5 The classic anti-pattern

```

# ANTI-PATTERN: an action inside a loop recomputes everything each iteration
for aid in aircraft_ids:
    print(df.filter(F.col("aircraft_id") == aid).count()) # N full scans!

# BETTER: one grouped action over the whole dataset
df.groupBy("aircraft_id").count().show()                # one pass

```

### ✏ Lab 6.1 — Prove laziness

Build a 4-step chain ending in `count()`, then add a second action and watch the Jobs tab show two jobs. Add `.cache()` + a warm-up count and confirm the second action's job reads from cache (much shorter duration).

### 🔑 Key takeaways

- Transformations are lazy; actions trigger the whole lineage.
- Narrow = no movement; wide = shuffle across the network (new stage).
- Filter/select early to shrink data before shuffles.
- Cache only reused DataFrames that fit in memory; unpersist when done.
- Never put an action inside a loop — aggregate in one pass.

# 07

ANALYTICS

## Joins, Aggregations & Window Functions

Combine datasets, summarize with `groupBy`, and compute per-row analytics with windows — the analyst's core toolkit.



# PySpark Joins, Aggregations & Window Functions

These three operations answer most analytical questions. They are also where shuffles live, so correctness and performance go hand in hand.

## 🎯 Learning objectives

- Apply inner, outer, semi and anti joins
- Use broadcast joins to avoid shuffles
- Summarize data with groupBy and agg
- Compute running and ranked metrics with windows
- Pick groupBy vs window correctly

## 7.1 Joins

We enrich telemetry (the fact table) with an aircraft dimension (model, operator, home base).

### Join semantics (L = left table, R = right table)



*PySpark also supports left\_semi (filter L by match) and left\_anti (filter L by non-match) — no R columns returned.*

**Figure 7.1.** Join semantics. Inner keeps matches; left/right keep all rows from one side; full outer keeps everything with nulls where there is no match.

```
fact = spark.read.parquet("/raw/aircraft_telemetry")           # many rows / aircraft
dim = spark.createDataFrame([
    ("A320-001", "A320", "IndiGo", "BLR"),
    ("A320-002", "A320", "Vistara", "DEL"),
    ("A350-010", "A350", "Air India", "BOM"),
], ["aircraft_id", "model", "operator", "home_base"])

joined = fact.join(dim, on="aircraft_id", how="inner")         # inner join on key
joined.select("aircraft_id", "model", "operator", "altitude_ft").show(3)
```

#### CONSOLE OUTPUT

```
+-----+-----+-----+-----+
|aircraft_id|model|operator|altitude_ft|
+-----+-----+-----+-----+
|  A320-001| A320|  IndiGo|      38000|
|  A320-002| A320| Vistara|      41000|
|  A350-010| A350|Air India|      39000|
+-----+-----+-----+-----+
```

### Broadcast the small side

When one table is small (a dimension), broadcast it so Spark ships a copy to every executor and skips the shuffle entirely. Use `fact.join(F.broadcast(dim), "aircraft_id")`. AQE often does this automatically, but the explicit hint guarantees it.

| how=                | Keeps                                       | Use when                           |
|---------------------|---------------------------------------------|------------------------------------|
| inner               | only matching keys                          | both sides required                |
| left / left_outer   | all left + matched right                    | keep all facts, enrich if possible |
| right / right_outer | all right + matched left                    | keep all dimension rows            |
| full / outer        | everything, nulls fill gaps                 | reconcile two sources              |
| left_semi           | left rows that have a match (no right cols) | existence filter                   |
| left_anti           | left rows with no match                     | find orphans / missing             |

## 7.2 Aggregations with groupBy

`groupBy().agg()` collapses rows into per-group summaries.

```
from pyspark.sql import functions as F

summary = (joined.groupBy("operator", "model")
            .agg(F.count("*").alias("readings"),
                 F.round(F.avg("fuel_flow_kgph"), 1).alias("avg_fuel"),
                 F.max("engine_temp_c").alias("max_temp"),
                 F.expr("percentile_approx(altitude_ft, 0.5)").alias("median_alt"))
            .orderBy(F.desc("avg_fuel")))
summary.show()
```

#### CONSOLE OUTPUT

```
+-----+-----+-----+-----+-----+-----+
|operator|model|readings|avg_fuel|max_temp|median_alt|
+-----+-----+-----+-----+-----+-----+
|Air India|A350|   18420|  2410.6|    655|   39000|
| Vistara|A320|   15903|  1980.3|    628|   41000|
|  IndiGo|A320|   16110|  1875.9|    612|   38000|
+-----+-----+-----+-----+-----+-----+
```

## 7.3 Window functions

A **window** computes a value for each row relative to a group of related rows — **without collapsing** them. This is how you rank, compute running totals, or compare a reading to the

previous one.

### Window function — partition, order, then compute per row (rows are kept)



**Figure 7.2.** A window partitions and orders rows, then computes per row. Unlike groupBy, the row count is preserved and a new column is appended.

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Partition per aircraft, ordered by time
w = Window.partitionBy("aircraft_id").orderBy("ts")

trend = (fact
    .withColumn("rn", F.row_number().over(w))
    .withColumn("prev_alt", F.lag("altitude_ft", 1).over(w))
    .withColumn("alt_delta", F.col("altitude_ft") - F.col("prev_alt"))
    # running 3-reading average fuel flow
    .withColumn("fuel_ma3",
        F.avg("fuel_flow_kgph").over(w.rowsBetween(-2, 0))))
trend.select("aircraft_id", "ts", "altitude_ft", "alt_delta", "fuel_ma3").show(4)
```

#### CONSOLE OUTPUT

```
+-----+-----+-----+-----+
|aircraft_id|ts|altitude_ft|alt_delta|fuel_ma3|
+-----+-----+-----+-----+
|A320-001|2026-06-01 08:00:00|38000|null|1850.0|
|A320-001|2026-06-01 08:05:00|38500|500|1862.5|
|A320-001|2026-06-01 08:10:00|39000|500|1875.9|
|A320-001|2026-06-01 08:15:00|38800|-200|1880.2|
+-----+-----+-----+-----+
```

| Window function                | Gives                                  |
|--------------------------------|----------------------------------------|
| row_number()                   | unique sequential number per partition |
| rank() / dense_rank()          | ranking with / without gaps on ties    |
| lag() / lead()                 | previous / next row's value            |
| sum/avg ... .over(rowsBetween) | running totals & moving averages       |

### **groupBy vs window**

Use **groupBy** when you want *one row per group* (a summary). Use a **window** when you want to keep every row but add a metric computed across its group (rank, delta, running average).

### **SkyOps — detecting anomalies**

Predictive-maintenance teams use windows to flag when engine temperature rises faster than its recent moving average for the *same* aircraft. `lag()` gives the previous reading, a rolling `avg().over(rowsBetween(-5,0))` gives the trend, and a threshold flags the anomaly — all while keeping every original reading for audit.

### **Lab 7.1 — Top reading per aircraft**

Use `row_number()` over a window partitioned by `aircraft_id` ordered by `fuel_flow_kgph desc`, then filter `rn == 1` to get each aircraft's highest-fuel reading. Compare with a broadcast-joined dimension to label the operator.

### **Key takeaways**

- Pick the join type deliberately; broadcast small dimensions to skip shuffles.
- `groupBy().agg()` collapses rows into summaries.
- Windows compute per-row metrics without collapsing rows.
- `lag/lead`, ranking and rolling windows drive trend & anomaly analytics.
- `groupBy` = one row per group; `window` = keep rows, add a column.

# 08

STREAMING

## Introduction to Real-Time Data Streaming

Batch vs streaming, the unbounded-table model, micro-batches, output modes, watermarks and exactly-once.



# Introduction to Real-Time Data Streaming in PySpark

Structured Streaming reuses everything you already know: the same DataFrame API, now over data that never stops arriving.

## Learning objectives

- Contrast batch and streaming processing
- Explain the unbounded-table / micro-batch model
- Build a streaming query with readStream/writeStream
- Choose append, update or complete output mode
- Handle late data with event time and watermarks

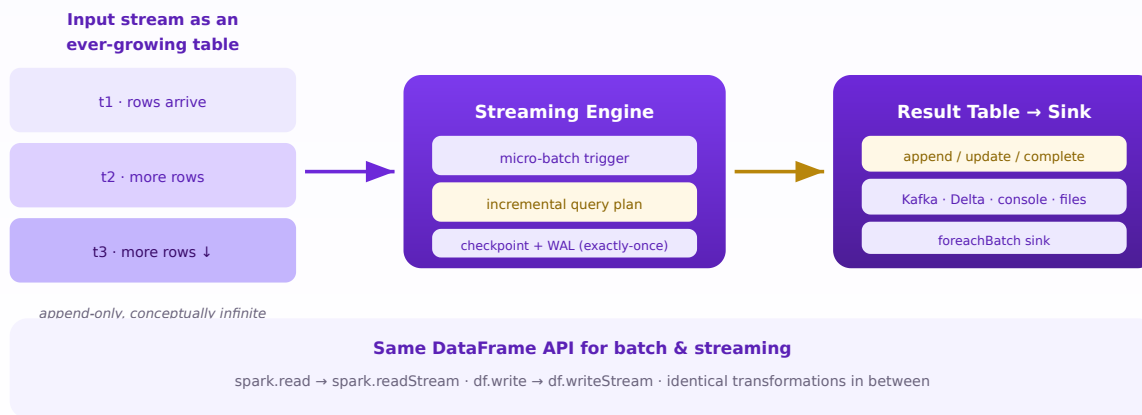
## 8.1 Batch vs streaming

|         | Batch                                           | Streaming                                                   |
|---------|-------------------------------------------------|-------------------------------------------------------------|
| Input   | Finite dataset                                  | Unbounded, continuous                                       |
| Latency | Minutes-hours                                   | Seconds (near real-time)                                    |
| Trigger | You run it                                      | Runs continuously                                           |
| API     | <code>spark.read</code> / <code>df.write</code> | <code>spark.readStream</code> / <code>df.writeStream</code> |

## 8.2 The unbounded-table model

Structured Streaming treats a live stream as a table that keeps growing. Each **micro-batch** processes the new rows since the last batch and incrementally updates a result table, which is written to a **sink**. Crucially, the same transformations work on this streaming DataFrame as on a static one.

## Structured Streaming — unbounded table & micro-batch model



**Figure 8.1.** Structured Streaming model. New rows append to a conceptually infinite table; the engine processes them in micro-batches and emits results to a sink, with checkpointing for exactly-once.

### 8.3 A first streaming query

We start simple with the built-in `rate` source (synthetic rows) to learn the shape, then switch to a real source in Module 9.

```
PYSPARK / PYTHON

from pyspark.sql import functions as F

# readStream instead of read – everything else is familiar DataFrame code
stream = (spark.readStream
          .format("rate")           # synthetic source: 1 row/sec with a timestamp
          .option("rowsPerSecond", 5)
          .load())

agg = (stream
      .withColumn("aircraft", F.concat(F.lit("A320-"),
                                       (F.col("value") % 3).cast("string")))
      .groupBy("aircraft")
      .count())

query = (agg.writeStream
        .outputMode("complete")    # full result table each trigger
        .format("console")
        .option("truncate", False)
        .start())

query.awaitTermination(15)        # run ~15s then stop
```

## CONSOLE OUTPUT

```
-----  
Batch: 3  
-----  
+-----+-----+  
|aircraft|count|  
+-----+-----+  
|A320-0  |    5 |  
|A320-1  |    5 |  
|A320-2  |    5 |  
+-----+-----+
```

## 8.4 Output modes

| Mode     | Emits                        | Use with                                 |
|----------|------------------------------|------------------------------------------|
| append   | only new rows                | non-aggregated / windowed-with-watermark |
| update   | rows that changed this batch | running aggregations                     |
| complete | the entire result table      | small aggregations / dashboards          |

## 8.5 Event time & watermarks

Real sensor data arrives **late** and out of order. Streaming aggregations should use **event time** (when the reading happened) not processing time. A **watermark** tells Spark how long to wait for late data before finalizing a window — bounding state so it does not grow forever.

```
----- PYSARK / PYTHON -----  
  
windowed = (stream_with_ts  
    .withWatermark("event_time", "10 minutes")           # tolerate 10 min lateness  
    .groupBy(F.window("event_time", "5 minutes"),        # 5-min tumbling windows  
             "aircraft_id")  
    .agg(F.avg("fuel_flow_kgph").alias("avg_fuel")))
```

### Why watermarks matter

Without a watermark, Spark must keep *all* window state in memory forever in case a late row arrives. The watermark lets Spark drop state older than the threshold — the difference between a stream that runs for months and one that slowly runs out of memory.

### Checkpointing is mandatory for reliability

Always set `.option("checkpointLocation", "/path")` on production streams. The checkpoint stores offsets and aggregation state so the query can recover exactly where it left off after a restart — this is what enables exactly-once processing.

## 8.6 Managing a streaming query

```
----- PYSARK / PYTHON -----  
  
print(query.id)           # stable id across restarts  
print(query.status)       # is it processing? waiting for data?  
print(query.recentProgress) # throughput, batch duration, input rows  
query.stop()              # graceful shutdown
```



### Lab 8.1 — Your first stream

Run the rate-source query above, watch batches print, then add

`.option("checkpointLocation")` and a 30-second window aggregation. Stop and restart the query and confirm it resumes from the checkpoint.

#### Key takeaways

- Streaming reuses the DataFrame API: `readStream` / `writeStream`.
- A stream is an unbounded table processed in micro-batches.
- Output modes: append, update, complete — match them to your query.
- Use event time + watermarks to handle late, out-of-order data and bound state.
- Checkpointing gives fault tolerance and exactly-once semantics.

# 09

INTEGRATION

## Streaming Architecture & Kafka Integration

A production streaming architecture and a complete Kafka →  
Spark → Delta pipeline, end to end.



# Streaming Architecture & Integration with Kafka and Other Sources

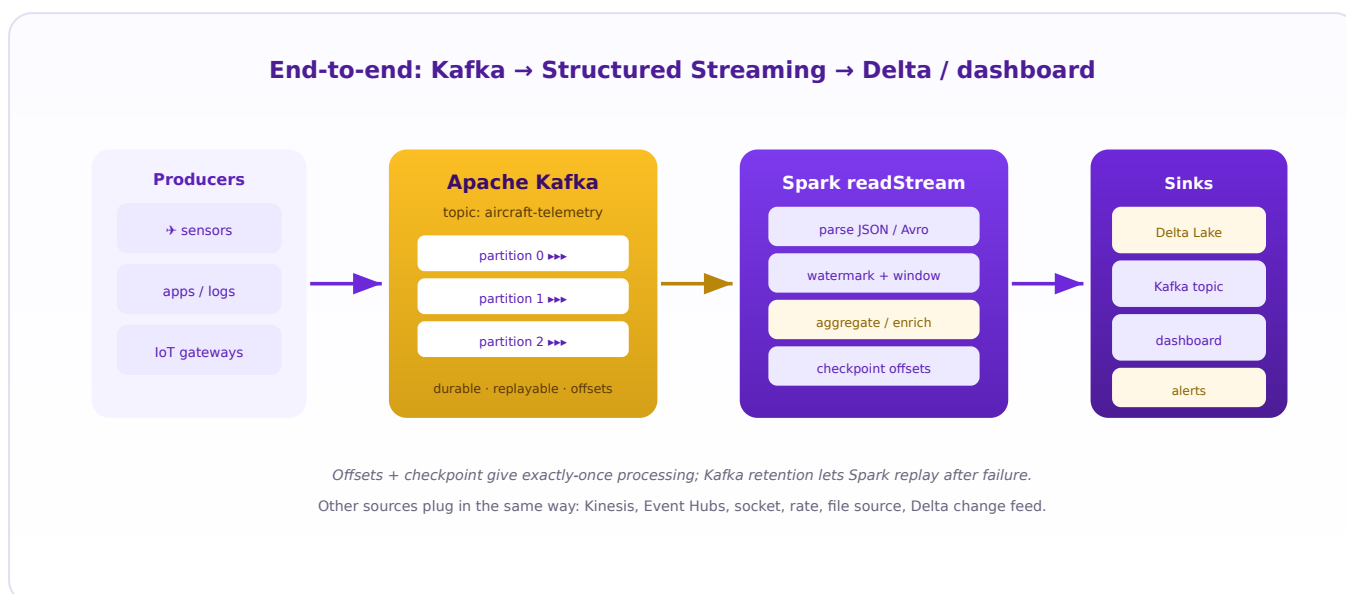
We connect the pieces into a real-time pipeline: a durable message bus feeding Spark, which enriches, aggregates and lands results for dashboards and alerts.

## Learning objectives

- Describe a production streaming architecture
- Read a Kafka topic with Structured Streaming
- Parse and enrich streaming JSON
- Window-aggregate and write to a sink
- Recognize other sources that plug in identically

## 9.1 Why Kafka in front of Spark

**Apache Kafka** is a distributed, durable, replayable log. Putting it between producers (sensors, apps) and Spark decouples them: producers never block on the consumer, data is buffered and retained, and Spark can **replay** from stored offsets after a failure. Topics are split into **partitions** for parallelism — Spark assigns partitions to tasks much like file partitions.



**Figure 9.1.** End-to-end streaming architecture: producers → Kafka topic (partitioned, durable) → Spark readStream (parse, watermark, aggregate) → sinks (Delta, Kafka, dashboard, alerts).

## 9.2 Reading from Kafka

The Kafka source returns a fixed schema; the payload lives in the binary `value` column, which you cast and parse.

```
raw = (spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "broker1:9092,broker2:9092")
      .option("subscribe", "aircraft-telemetry")
      .option("startingOffsets", "latest")           # or "earliest" to replay
      .load())

# Kafka gives: key, value (bytes), topic, partition, offset, timestamp
raw.printSchema()
```

#### CONSOLE OUTPUT

```
root
|-- key: binary (nullable = true)
|-- value: binary (nullable = true)
|-- topic: string (nullable = true)
|-- partition: integer (nullable = true)
|-- offset: long (nullable = true)
|-- timestamp: timestamp (nullable = true)
```

### 9.3 Parsing the JSON payload

```
from pyspark.sql import functions as F
from pyspark.sql.types import (StructType, StructField,
                               StringType, DoubleType, TimestampType)

payload = StructType([
    StructField("aircraft_id", StringType()),
    StructField("event_time", TimestampType()),
    StructField("altitude_ft", DoubleType()),
    StructField("fuel_flow_kgph", DoubleType()),
    StructField("engine_temp_c", DoubleType()),
])

events = (raw
        .select(F.from_json(F.col("value").cast("string"), payload).alias("d"))
        .select("d.*"))           # flatten the struct into top-level columns
```

### 9.4 Windowed aggregation with watermark

```
fleet_kpis = (events
    .withWatermark("event_time", "2 minutes")
    .groupBy(F.window("event_time", "1 minute"), "aircraft_id")
    .agg(F.avg("fuel_flow_kgph").alias("avg_fuel"),
        F.max("engine_temp_c").alias("max_temp"),
        F.count("*").alias("readings"))
    # flag a possible overheat for downstream alerting
    .withColumn("overheat", F.col("max_temp") > 650))
```

### 9.5 Writing to a sink (Delta) + back to Kafka

```
# Sink 1: durable analytics table (Delta), exactly-once via checkpoint
delta_q = (fleet_kpis.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/chk/fleet_kpis")
    .start("/clean/fleet_kpis"))

# Sink 2: publish overhear alerts back to a Kafka topic for ops systems
alerts = (fleet_kpis.filter("overheat")
    .select(F.col("aircraft_id").cast("string").alias("key"),
        F.to_json(F.struct("*")).alias("value")))

alert_q = (alerts.writeStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker1:9092")
    .option("topic", "fleet-alerts")
    .option("checkpointLocation", "/chk/fleet_alerts")
    .start())

spark.streams.awaitAnyTermination()
```

### Run from the shell with the Kafka package

The Kafka connector is an external package. Start your app with it, e.g. `spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 app.py` (match the Spark/Scala version).

## 9.6 Other sources plug in the same way

| Source              | format()                                        | Notes                            |
|---------------------|-------------------------------------------------|----------------------------------|
| Apache Kafka        | <code>kafka</code>                              | durable, replayable, partitioned |
| AWS Kinesis         | <code>kinesis</code>                            | managed AWS stream               |
| Azure Event Hubs    | <code>eventhubs</code> / <code>kafka</code>     | Kafka-compatible endpoint        |
| Files (landing dir) | <code>cloudFiles</code> / <code>json/csv</code> | Auto Loader picks up new files   |
| Delta change feed   | <code>delta</code>                              | stream changes from a table      |
| Socket / rate       | <code>socket</code> / <code>rate</code>         | testing only                     |

### SkyOps — the full picture

Aircraft sensors publish JSON to the `aircraft-telemetry` Kafka topic. Spark parses it, computes per-minute fuel and temperature KPIs per aircraft with a 2-minute watermark, lands them in a Delta table powering the fleet dashboard, and simultaneously publishes overhear alerts to `fleet-alerts` for the operations team — the same DataFrame skills from Modules 4–7, now continuous.

### Lab 9.1 — End-to-end pipeline

Spin up a local Kafka (or use the `rate` source as a stand-in producer), produce a few JSON telemetry events, run the `parse` → `watermark` → `window` → Delta pipeline, then query the Delta output as a batch DataFrame to confirm rows landed. Bonus: trigger an `overheat` and verify the alert topic receives it.

#### Key takeaways

- Kafka decouples producers from Spark and enables durable, replayable ingestion.
- The Kafka source exposes `value` as bytes — cast and parse with `from_json`.
- Combine watermark + windowed aggregation for correct real-time KPIs.
- Use checkpoints for exactly-once; write to Delta and/or back to Kafka.
- Kinesis, Event Hubs, Auto Loader and Delta CDF integrate with the identical API.